

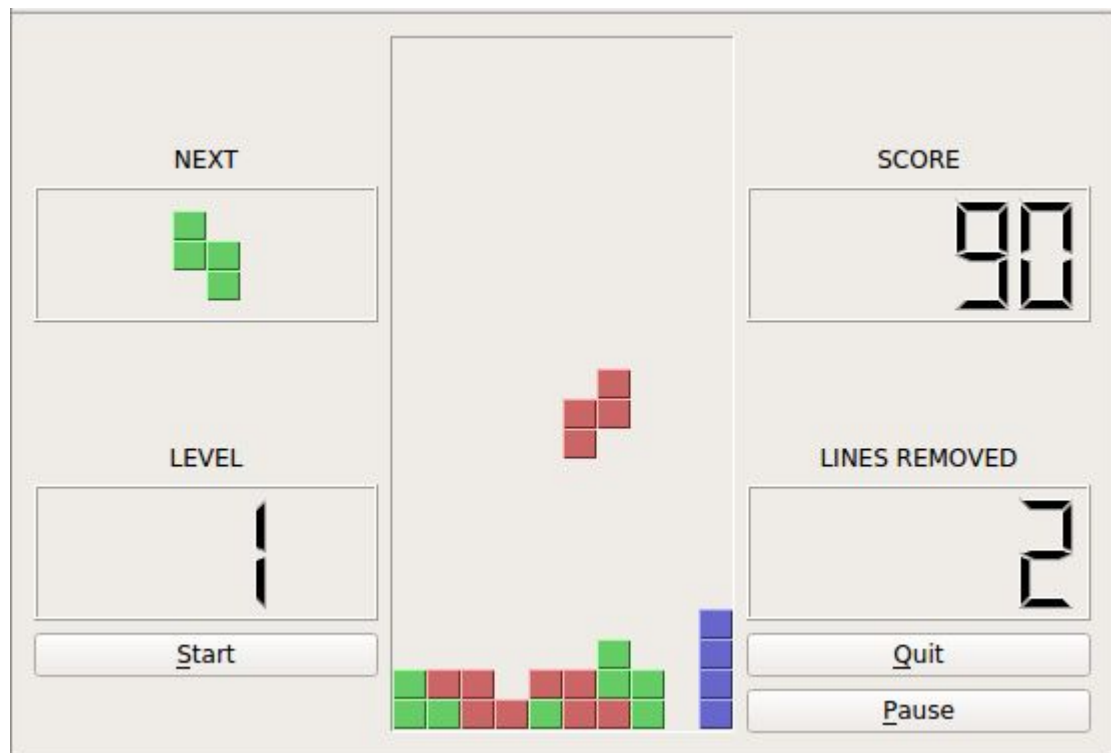
PS10: QT Programming Tetrix Implementation



CMPE 230 - Spring 2026

Gökçe Uludoğan

Tetrix Game



Implementation

- Created using only three classes:
 - TetrixWindow
 - Displays the player's score, number of lives, and information about the next piece to appear.
 - TetrixBoard
 - Contains the game logic, handles keyboard input, and displays the pieces on the playing area.
 - TetrixPiece
 - Contains information about each piece.

TetrixWindow

Class Definition

- Display the game information and contains the playing area
 - variables for the board,
 - various display widgets,
 - buttons to allow the user to start a new game, pause the current game, and quit.

```
class TetrixWindow : public QWidget
{
    Q_OBJECT

public:
    TetrixWindow(QWidget *parent = nullptr);

private:
    QLabel *createLabel(const QString &text);

    TetrixBoard *board;
    QLabel *nextPieceLabel;
    QLCDNumber *scoreLcd;
    QLCDNumber *levelLcd;
    QLCDNumber *linesLcd;
    QPushButton *startButton;
    QPushButton *quitButton;
    QPushButton *pauseButton;
};
```

TetrixWindow

Class Implementation - Constructor

- Constructor
 - Constructing a **TetrixBoard** instance for the playing area
 - **QLCDNumber**
 - To display the score
 - Buttons
 - Start
 - Quit
 - Pause
 - never receive the keyboard focus: TetrixBoard receives all the keyboard events

```
TetrixWindow::TetrixWindow(QWidget *parent)
    : QWidget(parent), board(new TetrixBoard)
{
```

```
    scoreLcd = new QLCDNumber(5);
    scoreLcd->setSegmentStyle(QLCDNumber::Filled);
```

```
    startButton = new QPushButton(tr("&Start"));
    startButton->setFocusPolicy(Qt::NoFocus);
    quitButton = new QPushButton(tr("&Quit"));
    quitButton->setFocusPolicy(Qt::NoFocus);
    pauseButton = new QPushButton(tr("&Pause"));
    pauseButton->setFocusPolicy(Qt::NoFocus);
```

TetrixWindow

Class Implementation - Constructor

- Connect button signals

```
connect(startButton, &QPushButton::clicked, board, &TetrixBoard::start);  
connect(quitButton , &QPushButton::clicked, qApp, &QCoreApplication::quit);  
connect(pauseButton, &QPushButton::clicked, board, &TetrixBoard::pause);
```

- Connect board signals

```
connect(board, &TetrixBoard::scoreChanged,  
        scoreLcd, QOverload<int>::of(&QLCDNumber::display));  
connect(board, &TetrixBoard::levelChanged,  
        levelLcd, QOverload<int>::of(&QLCDNumber::display));  
connect(board, &TetrixBoard::linesRemovedChanged,  
        linesLcd, QOverload<int>::of(&QLCDNumber::display));
```

TetrixWindow

Class Implementation - Constructor

- QGridLayout
 - To place the label, LCD widgets, and the board

```
QGridLayout *layout = new QGridLayout;
layout->addWidget(createLabel(tr("NEXT")), 0, 0);
layout->addWidget(nextPieceLabel, 1, 0);
layout->addWidget(createLabel(tr("LEVEL")), 2, 0);
layout->addWidget(levelLcd, 3, 0);
layout->addWidget(startButton, 4, 0);
layout->addWidget(board, 0, 1, 6, 1);
layout->addWidget(createLabel(tr("SCORE")), 0, 2);
layout->addWidget(scoreLcd, 1, 2);
layout->addWidget(createLabel(tr("LINES REMOVED")), 2, 2);
layout->addWidget(linesLcd, 3, 2);
layout->addWidget(quitButton, 4, 2);
layout->addWidget(pauseButton, 5, 2);
setLayout(layout);

setWindowTitle(tr("Tetrix"));
resize(550, 370);
}
```

TetrixWindow

Class Implementation - createLabel()

```
QLabel *TetrixWindow::createLabel(const QString &text)
{
    QLabel *label = new QLabel(text);
    label->setAlignment(Qt::AlignHCenter | Qt::AlignBottom);
    return label;
}
```


TetrixPiece

Class Definition

- Holds information about a piece in the game's playing area,
 - its shape,
 - position,
 - the range of positions it can occupy on the board
- Each shape contains four blocks, defined by the **coords**
- Each piece has a high-level description that is stored internally in the **pieceShape**.

```
enum TetrixShape { NoShape, ZShape, SShape, LineShape,  
                  TShape, SquareShape, LShape,  
                  MirroredLShape };
```

```
class TetrixPiece  
{  
public:  
    TetrixPiece() { setShape(NoShape); }  
  
    void setRandomShape();  
    void setShape(TetrixShape shape);  
  
    TetrixShape shape() const { return pieceShape; }  
    int x(int index) const { return coords[index][0]; }  
    int y(int index) const { return coords[index][1]; }  
    int minX() const;  
    int maxX() const;  
    int minY() const;  
    int maxY() const;  
    TetrixPiece rotatedLeft() const;  
    TetrixPiece rotatedRight() const;  
  
private:  
    void setX(int index, int x) { coords[index][0] = x; }  
    void setY(int index, int y) { coords[index][1] = y; }  
  
    TetrixShape pieceShape;  
    int coords[4][2];  
};
```

TetrixPiece

Class Implementation

- setRandomShape()
 - Chooses a random shape from the TetrixShape enum
- setShape()
 - uses a look-up table of pieces to associate each shape with an array of block positions

```
void TetrixPiece::setRandomShape()
{
    setShape(TetrixShape(QRandomGenerator::global()->bounded(7) + 1));
}
```

```
void TetrixPiece::setShape(TetrixShape shape)
{
    static constexpr int coordsTable[8][4][2] = {
        { { 0, 0 }, { 0, 0 }, { 0, 0 }, { 0, 0 } },
        { { 0, -1 }, { 0, 0 }, { -1, 0 }, { -1, 1 } },
        { { 0, -1 }, { 0, 0 }, { 1, 0 }, { 1, 1 } },
        { { 0, -1 }, { 0, 0 }, { 0, 1 }, { 0, 2 } },
        { { -1, 0 }, { 0, 0 }, { 1, 0 }, { 0, 1 } },
        { { 0, 0 }, { 1, 0 }, { 0, 1 }, { 1, 1 } },
        { { -1, -1 }, { 0, -1 }, { 0, 0 }, { 0, 1 } },
        { { 1, -1 }, { 0, -1 }, { 0, 0 }, { 0, 1 } }
    };

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 2; ++j)
            coords[i][j] = coordsTable[shape][i][j];
    }
    pieceShape = shape;
}
```

TetrixPiece

Class Implementation

- minX(), maxX(), minY(), maxY()

return the minimum and maximum coordinates occupied by the blocks

```
int TetrixPiece::minY() const
{
    int min = coords[0][1];
    for (int i = 1; i < 4; ++i)
        min = qMin(min, coords[i][1]);
    return min;
}

int TetrixPiece::maxY() const
{
    int max = coords[0][1];
    for (int i = 1; i < 4; ++i)
        max = qMax(max, coords[i][1]);
    return max;
}
```

```
int TetrixPiece::minX() const
{
    int min = coords[0][0];
    for (int i = 1; i < 4; ++i)
        min = qMin(min, coords[i][0]);
    return min;
}

int TetrixPiece::maxX() const
{
    int max = coords[0][0];
    for (int i = 1; i < 4; ++i)
        max = qMax(max, coords[i][0]);
    return max;
}
```

TetrixPiece

Class Implementation

- rotatedLeft(), rotatedRight()
 - returns a new piece with the same shape as an existing piece, but rotated

```
TetrixPiece TetrixPiece::rotatedLeft() const
{
    if (pieceShape == SquareShape)
        return *this;

    TetrixPiece result;
    result.pieceShape = pieceShape;
    for (int i = 0; i < 4; ++i) {
        result.setX(i, y(i));
        result.setY(i, -x(i));
    }
}
```

```
TetrixPiece TetrixPiece::rotatedRight() const
{
    if (pieceShape == SquareShape)
        return *this;

    TetrixPiece result;
    result.pieceShape = pieceShape;
    for (int i = 0; i < 4; ++i) {
        result.setX(i, -y(i));
        result.setY(i, x(i));
    }
}
```

TetrixBoard

Class Definition

- Contains the game logic and display features
- Slots
 - start()
 - pause()
- Signals
 - scoreChanged()
 - levelChanged()
 - linesRemovedChanged()
- Events
 - paintEvent()
 - keyPressEvent()
 - timerEvent()

```
class TetrixBoard : public QFrame
{
    Q_OBJECT

public:
    TetrixBoard(QWidget *parent = nullptr);

    void setNextPieceLabel(QLabel *label);
    QSize sizeHint() const override;
    QSize minimumSizeHint() const override;

public slots:
    void start();
    void pause();

signals:
    void scoreChanged(int score);
    void levelChanged(int level);
    void linesRemovedChanged(int numLines);

protected:
    void paintEvent(QPaintEvent *event) override;
    void keyPressEvent(QKeyEvent *event) override;
    void timerEvent(QTimerEvent *event) override;
```


TetrixBoard

Class Definition

- The rest is the reimplementations of protected event handlers and private functions:

```
private:
    enum { BoardWidth = 10, BoardHeight = 22 };

    TetrixShape &shapeAt(int x, int y) { return board[(y * BoardWidth) + x]; }
    int timeoutTime() { return 1000 / (1 + level); }
    int squareWidth() { return contentsRect().width() / BoardWidth; }
    int squareHeight() { return contentsRect().height() / BoardHeight; }
    void clearBoard();
    void dropDown();
    void oneLineDown();
    void pieceDropped(int dropHeight);
    void removeFullLines();
    void newPiece();
    void showNextPiece();
    bool tryMove(const TetrixPiece &newPiece, int newX, int newY);
    void drawSquare(QPainter &painter, int x, int y, TetrixShape shape);
```

```
QBasicTimer timer;
QPointer<QLabel> nextPieceLabel;
bool isStarted;
bool isPaused;
bool isWaitingAfterLine;
TetrixPiece curPiece;
TetrixPiece nextPiece;
int curX;
int curY;
int numLinesRemoved;
int numPiecesDropped;
int score;
int level;
TetrixShape board[BoardWidth * BoardHeight];
};
```

TetrixBoard

Class Implementation

- Constructor
 - customizes the frame style of the widget
 - Ensures that keyboard input will be received by the widget by using Qt::StrongFocus for the focus policy
 - Initializes the game state
- setNextPieceLabel()
 - To pass in an externally-constructed label to the board

```
TetrixBoard::TetrixBoard(QWidget *parent)
    : QFrame(parent), isStarted(false), isPaused(false)
{
    setFrameStyle(QFrame::Panel | QFrame::Sunken);
    setFocusPolicy(Qt::StrongFocus);
    clearBoard();

    nextPiece.setRandomShape();
}
```

```
void TetrixBoard::setNextPieceLabel(QLabel *label)
{
    nextPieceLabel = label;
}
```


TetrixBoard

Class Implementation - Slots

- start() slot
 - Resets the game's state, the player's score and level, and the contents of the board
 - Also emits signals to inform other components of these changes
 - Creates a new piece that is ready to be dropped into the playing area
 - Starts the timer that determines how often the piece drops down one row on the board.

```
void TetrixBoard::start()
{
    if (isPaused)
        return;

    isStarted = true;
    isWaitingAfterLine = false;
    numLinesRemoved = 0;
    numPiecesDropped = 0;
    score = 0;
    level = 1;
    clearBoard();

    emit linesRemovedChanged(numLinesRemoved);
    emit scoreChanged(score);
    emit levelChanged(level);

    newPiece();
    timer.start(timeoutTime(), this);
}
```

TetrixBoard

Class Implementation - Slots

- pause() slot
 - Stops the internal timer

```
void TetrixBoard::pause()
{
    if (!isStarted)
        return;

    isPaused = !isPaused;
    if (isPaused) {
        timer.stop();
    } else {
        timer.start(timeoutTime(), this);
    }
    update();
}
```

TetrixBoard

Class Implementation - Events

- paintEvent()
 - Calling the base class's implementation of paintEvent()
 - Constructs a QPainter for use on the board
 - For each space on the board that is occupied by a piece, call the drawSquare() to draw a block at that position.
 - Current piece is drawn block-by-block at its current position

```
void TetrixBoard::paintEvent(QPaintEvent *event)
{
```

```
    QFrame::paintEvent(event);
```

```
    QPainter painter(this);
```

```
    QRect rect = contentsRect();
```

```
    int boardTop = rect.bottom() - BoardHeight*squareHeight();
```

```
    for (int i = 0; i < BoardHeight; ++i) {
```

```
        for (int j = 0; j < BoardWidth; ++j) {
```

```
            TetrixShape shape = shapeAt(j, BoardHeight - i - 1);
```

```
            if (shape != NoShape)
```

```
                drawSquare(painter, rect.left() + j * squareWidth(),  
                           boardTop + i * squareHeight(), shape);
```

```
        }
```

```
    }
```

```
    if (curPiece.shape() != NoShape) {
```

```
        for (int i = 0; i < 4; ++i) {
```

```
            int x = curX + curPiece.x(i);
```

```
            int y = curY - curPiece.y(i);
```

```
            drawSquare(painter, rect.left() + x * squareWidth(),  
                       boardTop + (BoardHeight - y - 1) * squareHeight(),  
                       curPiece.shape());
```

TetrixBoard

Class Implementation - Events

- keyPressed()
 - Called whenever the player presses a key while the TetrixBoard widget has the keyboard focus.
 - Passes on the event to the base class if there is no current game, the game is running but paused, or if there is no current shape to control
 - Otherwise, calls the relevant function to handle the input

```
void TetrixBoard::keyPressEvent(QKeyEvent *event)
{
    if (!isStarted || isPaused || curPiece.shape() == NoShape) {
        QFrame::keyPressEvent(event);
        return;
    }
}
```

```
switch (event->key()) {
case Qt::Key_Left:
    tryMove(curPiece, curX - 1, curY);
    break;
case Qt::Key_Right:
    tryMove(curPiece, curX + 1, curY);
    break;
case Qt::Key_Down:
    tryMove(curPiece.rotatedRight(), curX, curY);
    break;
case Qt::Key_Up:
    tryMove(curPiece.rotatedLeft(), curX, curY);
    break;
case Qt::Key_Space:
    dropDown();
    break;
case Qt::Key_D:
    oneLineDown();
    break;
default:
    QFrame::keyPressEvent(event);
}
```

TetrixBoard

Class Implementation - Events

- timerEvent()
 - Called every time the class's QBasicTimer instance times out.
 - If a row (or line) has just been filled, creates a new piece and resets the timer
 - Otherwise, moves the current piece down by one row.

```
void TetrixBoard::timerEvent(QTimerEvent *event)
{
    if (event->timerId() == timer.timerId()) {
        if (isWaitingAfterLine) {
            isWaitingAfterLine = false;
            newPiece();
            timer.start(timeoutTime(), this);
        } else {
            oneLineDown();
        }
    } else {
        QFrame::timerEvent(event);
    }
}
```

TetrixBoard

Class Implementation

- `clearBoard()`
 - Fills the board with `NoShape`
- `dropDown()`
 - moves the current piece down as far as possible on the board
- `oneLineDown()`
 - moves the current piece down by one row

```
void TetrixBoard::oneLineDown()
{
    if (!tryMove(curPiece, curX, curY - 1))
        pieceDropped(1);
}
```

```
void TetrixBoard::clearBoard()
{
    for (int i = 0; i < BoardHeight * BoardWidth; ++i)
        board[i] = NoShape;
}
```

```
void TetrixBoard::dropDown()
{
    int dropHeight = 0;
    int newY = curY;
    while (newY > 0) {
        if (!tryMove(curPiece, curX, newY - 1))
            break;
        --newY;
        ++dropHeight;
    }
    pieceDropped(dropHeight);
}
```

TetrixBoard

Class Implementation

- pieceDropped()
 - Awards points to the player for positioning the current piece
 - Checks for full rows on the board
 - If no lines have been removed, creates a new piece to replace the current one

```
void TetrixBoard::pieceDropped(int dropHeight)
{
    for (int i = 0; i < 4; ++i) {
        int x = curX + curPiece.x(i);
        int y = curY - curPiece.y(i);
        shapeAt(x, y) = curPiece.shape();
    }

    ++numPiecesDropped;
    if (numPiecesDropped % 25 == 0) {
        ++level;
        timer.start(timeoutTime(), this);
        emit levelChanged(level);
    }

    score += dropHeight + 7;
    emit scoreChanged(score);
    removeFullLines();

    if (!isWaitingAfterLine)
        newPiece();
}
```


TetrixBoard

Class Implementation

- removeFullLines()
 - Called each time a piece has been dropped
 - Scans the board from bottom to top, looking for blank spaces on each row.
 - Full lines are copied down and the game state is updated.
 - Relevant signals are emitted.

```
void TetrixBoard::removeFullLines()
{
    int numFullLines = 0;

    for (int i = BoardHeight - 1; i >= 0; --i) {
        bool lineIsFull = true;

        for (int j = 0; j < BoardWidth; ++j) {
            if (shapeAt(j, i) == NoShape) {
                lineIsFull = false;
                break;
            }
        }

        if (lineIsFull) {
            ++numFullLines;
            for (int k = i; k < BoardHeight - 1; ++k) {
                for (int j = 0; j < BoardWidth; ++j)
                    shapeAt(j, k) = shapeAt(j, k + 1);
            }
            for (int j = 0; j < BoardWidth; ++j)
                shapeAt(j, BoardHeight - 1) = NoShape;
        }
    }

    if (numFullLines > 0) {
        numLinesRemoved += numFullLines;
        score += 10 * numFullLines;
        emit linesRemovedChanged(numLinesRemoved);
        emit scoreChanged(score);

        timer.start(500, this);
        isWaitingAfterLine = true;
        curPiece.setShape(NoShape);
        update();
    }
}
```

TetrixBoard

Class Implementation

- newPiece()
 - places the next available piece at the top of the board
 - Creates a new piece with a random shape:

```
void TetrixBoard::newPiece()
{
    curPiece = nextPiece;
    nextPiece.setRandomShape();
    showNextPiece();
    curX = BoardWidth / 2 + 1;
    curY = BoardHeight - 1 + curPiece.minY();

    if (!tryMove(curPiece, curX, curY)) {
        curPiece.setShape(NoShape);
        timer.stop();
        isStarted = false;
    }
}
```

- showNextPiece()
 - Updates the label that shows the next piece to be dropped

```
void TetrixBoard::showNextPiece()
{
    if (!nextPieceLabel)
        return;

    int dx = nextPiece.maxX() - nextPiece.minX() + 1;
    int dy = nextPiece.maxY() - nextPiece.minY() + 1;

    QPixmap pixmap(dx * squareWidth(), dy * squareHeight());
    QPainter painter(&pixmap);
    painter.fillRect(pixmap.rect(), nextPieceLabel->palette().window());

    for (int i = 0; i < 4; ++i) {
        int x = nextPiece.x(i) - nextPiece.minX();
        int y = nextPiece.y(i) - nextPiece.minY();
        drawSquare(painter, x * squareWidth(), y * squareHeight(),
                    nextPiece.shape());
    }
    nextPieceLabel->setPixmap(pixmap);
}
```

TetrixBoard

Class Implementation

- tryMove()
 - determines if a piece can be positioned at the specified coordinates
 - If the spaces on the board that the piece needs to occupy are already occupied by other pieces, return false to indicate that the move has failed.
 - If the piece could be placed on the board at the desired location,
 - updates the current piece and its position, the widget,
 - returns true to indicate success.

```
bool TetrixBoard::tryMove(const TetrixPiece &newPiece, int newX, int newY)
{
    for (int i = 0; i < 4; ++i) {
        int x = newX + newPiece.x(i);
        int y = newY - newPiece.y(i);
        if (x < 0 || x >= BoardWidth || y < 0 || y >= BoardHeight)
            return false;
        if (shapeAt(x, y) != NoShape)
            return false;
    }
}
```

```
    curPiece = newPiece;
    curX = newX;
    curY = newY;
    update();
    return true;
}
```

TetrixBoard

Class Implementation

- drawSquare()
 - Draws the blocks that make up each piece using different colors for pieces with different shapes
 - Uses a look-up table that relates each shape to an RGB value
 - Uses the painter provided to draw the block at the specified coordinates

```
void TetrixBoard::drawSquare(QPainter &painter, int x, int y, TetrixShape shape)
{
    static constexpr QRgb colorTable[8] = {
        0x000000, 0xCC6666, 0x66CC66, 0x6666CC,
        0xCCCC66, 0xCC66CC, 0x66CCCC, 0xDAAA00
    };

    QColor color = colorTable[int(shape)];
    painter.fillRect(x + 1, y + 1, squareWidth() - 2, squareHeight() - 2,
                    color);

    painter.setPen(color.lighter());
    painter.drawLine(x, y + squareHeight() - 1, x, y);
    painter.drawLine(x, y, x + squareWidth() - 1, y);

    painter.setPen(color.darker());
    painter.drawLine(x + 1, y + squareHeight() - 1,
                    x + squareWidth() - 1, y + squareHeight() - 1);
    painter.drawLine(x + squareWidth() - 1, y + squareHeight() - 1,
                    x + squareWidth() - 1, y + 1);
}
```

References

- <https://doc.qt.io/qt-5/qtwidgets-widgets-tetrix-example.html>